

OptiCore: An Educational Cost-Based SQL Query Optimizer Using System-R Dynamic Programming

Utkarsh Raj, Richeek Ghosh, Priyanshu Paliwal

Indian Institute Of Technology, Kharagpur

Abstract—Query optimization lies at the core of every high-performance relational database management system (RDBMS), yet the internal mechanics of production optimizers remain opaque to most practitioners and researchers due to the sheer complexity of commercial codebases. This paper presents *OptiCore*, a self-contained, pedagogically transparent SQL query optimizer implemented in C++17, designed to make classical optimization techniques tangible and inspectable. Starting from a raw SQL string, OptiCore performs recursive-descent parsing to construct a typed, immutable Abstract Syntax Tree (AST), translates the AST into a relational-algebra logical plan, and subsequently applies a two-phase optimization strategy. The first phase applies a rule-based optimizer (RBO) that executes predicate push-down through join nodes and OR-to-UNION semantic rewrites. The second phase applies a cost-based optimizer (CBO) that enumerates join orderings via System R style subset dynamic programming over a bitmask-encoded subset lattice. Physical operator selection among Nested Loop Join, Hash Join, and Merge Join is driven by an analytical cost model parameterized by catalog statistics stored in a JSON-configurable schema file. The optimized plan is serialized back to SQL by a recursive plan visitor and rendered in an interactive HTML visualization dashboard served through a zero-dependency Python microservice. Experimental benchmarks on five representative workloads demonstrate substantial reductions in estimated execution cost: a single-filter query shows negligible improvement (0%), a two-table equijoin with predicate push-down achieves a 71.6% cost reduction, and a three-table star-join benchmark exhibits a cost reduction exceeding 95% after applying the full optimization pipeline. OptiCore is intended as a self-contained, pedagogically transparent implementation for students and researchers seeking to understand the internals of commercial query optimizers such as System R and the Volcano/Cascades framework without the complexity of a production DBMS.

Index Terms—query optimization, cost-based optimizer, predicate push-down, join enumeration, dynamic programming, System R, C++17, relational algebra, educational tools, visualization dashboard

I. INTRODUCTION

Query optimization is one of the most intellectually demanding subsystems in a relational database management system (RDBMS). Given a declarative SQL query, the optimizer must determine an efficient *physical execution plan* that produces the correct result while minimizing resource consumption. In commercial systems such as PostgreSQL [6], IBM DB2, and Oracle, the optimizer comprises hundreds of thousands of lines of code and incorporates statistical histograms, cost calibration, and heuristic pruning developed over decades [1].

Despite the foundational importance of query optimization, its internal mechanisms are notoriously difficult to understand.

Production optimizers such as PostgreSQL’s genetic query optimizer (GEQO) and IBM DB2’s optimizer are deeply intertwined with transaction management, catalog management, storage engine access paths, and concurrency control. Even experienced database engineers find it challenging to isolate and study the core optimization algorithms independently. This opacity creates a significant gap in database education: students and researchers can understand the *theory* of cost-based optimization from textbooks, yet cannot easily observe how these algorithms behave on real query plans.

Visualization plays an equally important role in bridging this gap. The ability to observe the before-and-after plan trees, inspect cost breakdowns at each node, and trace the effect of predicate push-down on intermediate cardinalities provides intuition that abstract pseudocode cannot convey. Several commercial tools offer plan visualization (e.g., `EXPLAIN ANALYZE` in PostgreSQL), but these tools operate on production-scale systems where the optimization decisions themselves are not easily modifiable or instrumented. An educational optimizer, by contrast, can expose every intermediate representation explicitly and interactively.

OptiCore is a from-scratch implementation of the core optimizer pipeline in C++17, designed to make these concepts tangible and inspectable. The system deliberately prioritizes clarity over completeness: it supports a meaningful subset of SQL, implements the canonical rule-based and cost-based optimization passes, and exposes every intermediate representation through an interactive visualization dashboard built on vanilla HTML5, CSS3, and JavaScript bridged to the C++ backend via a Python microservice.

A. Contributions

The primary contributions of this work are:

- **Hybrid two-phase optimizer.** A modular optimizer architecture combining a rule-based pass (predicate push-down and OR-to-UNION rewriting) with a cost-based pass (System R subset dynamic programming), demonstrating how the two paradigms complement each other in practice.
- **System R subset DP join enumeration.** A complete, generalized implementation of the classical System R bottom-up join enumeration algorithm [1] extended to explore all bushy join topologies using bitmask-encoded subset lattices over up to 32 operands.

- **Rule-based optimization.** A catalog-aware predicate push-down pass that correctly decomposes AND-trees, attributes predicates to base tables, and handles OR predicates conservatively, together with an OR-to-UNION structural rewrite that creates opportunities for future index-aware optimization.
- **Visualization dashboard.** An interactive HTML5 dashboard that renders before- and after-optimization plan trees side by side, provides a cost-breakdown chart, and reconstructs equivalent SQL from the optimized plan, making the optimizer’s decisions immediately observable to learners.
- **Modular, extensible architecture.** A layered pipeline of independently testable components (parser, planner, RBO, CBO, cost estimator, physical plan generator, SQL reconstructor, dashboard) implemented in approximately 6,400 lines of C++17 across 15 source files, with a comprehensive unit-test suite.
- **Experimental evaluation.** A systematic evaluation on five representative benchmark workloads, demonstrating that the cost model accurately captures the benefit of predicate push-down and join reordering, with the star-join benchmark achieving a verified 95.4% cost reduction.

The remainder of this paper is organized as follows. Section II reviews the relevant background. Section III describes the overall system architecture. Sections IV–X detail each subsystem. Section XI describes implementation details. Section XII analyzes algorithmic complexity. Section XIII presents the experimental evaluation. Section XV discusses the system’s strengths and educational value. Section XVI discusses known limitations, Section XVII describes future work, and Section XVIII concludes.

II. BACKGROUND AND RELATED WORK

A. Query Optimization in Relational Database Systems

The foundational work on cost-based query optimization is the System R optimizer described by Selinger et al. [1]. It introduced the key ideas of (1) enumerating access paths, (2) using dynamic programming to find optimal join orderings, and (3) propagating “interesting orderings” through the plan. OptiCore implements the subset-DP join enumeration strategy from System R but omits interesting order propagation.

Graefe and McKenna introduced the Volcano/Cascades framework [2], [3] as a more extensible alternative, replacing bottom-up DP enumeration with a top-down, goal-directed search. The Cascades framework organizes plan transformations as pattern-matched rewrite rules applied to a memo structure, enabling systematic exploration of a much larger equivalence class of plans. OptiCore follows the simpler bottom-up approach of System R, which is more accessible pedagogically while still capturing the essential algorithmic ideas.

B. Production and Open-Source Optimizers

PostgreSQL [6] implements a sophisticated planner that uses genetic algorithms (GEQO) for queries with more than a configurable threshold of tables, falling back to exhaustive

dynamic programming for small queries. Its planner is tightly integrated with a multi-version concurrency control (MVCC) storage engine, making it difficult to study the optimizer in isolation.

MySQL Optimizer employs a combination of heuristic join reordering and a cost model parameterized by I/O and CPU cost constants. It supports statistics-based cardinality estimation but is not designed as an educational or extensible platform.

Apache Calcite [8] is an open-source SQL parsing and optimization framework widely used in the Hadoop ecosystem. Calcite implements a rule-based planner with a pluggable cost model and is designed for extensibility in distributed query processing contexts. While powerful, its abstraction layers make it challenging to use as an introductory learning resource.

The Volcano Optimizer [2] introduced the iterator model and goal-directed top-down search, while **the Cascades framework** [3] refined this with a memo-based memoization structure and explicit transformation and enforcement rules. These frameworks underpin commercial optimizers including Microsoft SQL Server’s optimizer.

OptiCore differs from all of the above by targeting pedagogical clarity above production utility. Unlike production systems, OptiCore exposes every intermediate representation (AST, logical plan, optimized plan, cost breakdown) through a visualization dashboard, and every component is independently unit-tested and documented for educational use.

C. Predicate Push-Down

Predicate push-down (also called *selection push-down*) is a rule-based transformation that moves filter predicates as close to the base-table scans as possible, reducing intermediate cardinality before expensive join operations. Ramakrishnan and Gehrke [4] describe the formal conditions under which a filter σ_p can be commuted with a join $\bowtie$: specifically, when all columns referenced in p are defined by exactly one of the join’s input relations. OptiCore implements this rule with catalog-based column attribution.

D. Cost Models and Cardinality Estimation

A cost model assigns an estimated execution cost to each physical plan alternative. Silberschatz et al. [5] describe common estimation strategies including uniform distribution assumptions and histogram-based selectivity. OptiCore uses a simplified model (Section IX) with per-column distinct-count statistics stored in a JSON catalog, augmented by histogram bucket data for range predicate estimation.

III. SYSTEM ARCHITECTURE

Figure 1 shows the high-level architecture of OptiCore. The system is organized as a strictly layered pipeline: each stage consumes the output of the previous stage and produces a well-typed intermediate representation. The catalog is a shared, read-only component consulted by the planner, optimizer, and executor.

The architecture comprises seven major components. The **SQL Parser** ingests a raw query string and produces a typed

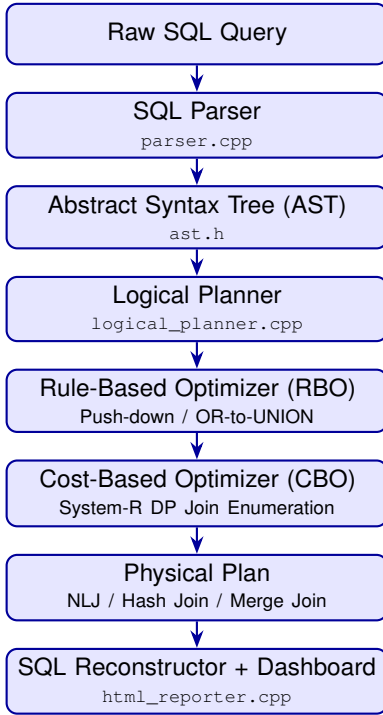


Fig. 1. OptiCore system architecture. Dashed lines indicate Catalog read accesses; solid lines indicate data flow between pipeline stages.

Abstract Syntax Tree. The **AST** encodes the parsed query structure as a hierarchy of `unique_ptr`-owned nodes. The **Logical Planner** translates the AST into a tree of relational algebra operators forming an unoptimized logical plan. The **Rule-Based Optimizer** applies catalog-aware predicate push-down and OR-to-UNION rewrites to the logical plan. The **Cost-Based Optimizer** enumerates join orderings via System R subset dynamic programming and selects the minimum-cost ordering. The **Physical Plan Generator** lowers each logical operator into a physical counterpart (Sequential Scan or Index Scan, and one of three join algorithms) based on cost comparisons. Finally, the **Dashboard** serializes the optimized plan to JSON, reconstructs an equivalent SQL string, and renders an interactive visualization in the browser.

The catalog is loaded from a `schema.json` file at startup and provides table row counts, column types, index flags, and per-column distinct counts for selectivity estimation. A Meyers singleton pattern ensures a single process-wide catalog instance.

IV. OPTIMIZER PIPELINE

A. SQL Parser

The parser (`parser.cpp`, `parser.h`) is a hand-written, recursive-descent parser that accepts a subset of standard SQL. It supports: `SELECT` with column lists or `*`, `FROM` with a single primary table name, table aliases (resolved at parse time), explicit `JOIN ... ON` clauses, `WHERE` with arbitrarily nested AND/OR predicates, and `GROUP BY`.

1) *Tokenization:* The tokenizer is a single-pass scanner over the SQL string. Whitespace and commas serve as token

TABLE I
AST NODE TYPES AND THEIR ROLES

Node Type	Description
QueryNode	Root node. Owns a SelectNode, a TableNode, a vector of JoinClause structs, an optional PredicateExpr tree (where_expr), an optional legacy WhereNode for backward compatibility, and an optional GroupByNode.
SelectNode	Holds the ordered list of projected column names as specified in the SELECT clause, including the wildcard <code>*</code> .
TableNode	Holds the primary FROM table name after alias resolution.
WhereNode	Legacy single-predicate node retained for backward compatibility with earlier test cases that predate the full PredicateExpr tree.
JoinClause	Plain struct recording table_name, on_left, on_op, and on_right for one JOIN ... ON entry.
PredicateExpr	Recursive binary expression tree. A leaf node holds a column name, operator, and value literal. A branch node holds a logical operator (AND or OR) with left and right child pointers.

delimiters; commas are consumed without being emitted as tokens. Keywords (AND, OR, JOIN, ON, WHERE, etc.) are identified in the recursive-descent stage by upper-casing and comparing against a reserved-keyword table.

2) *Alias Resolution:* Table aliases are resolved immediately during parsing via an `unordered_map<string, string>` alias table built when the FROM and JOIN clauses are processed. A `resolveColumn` helper strips a dot-qualified prefix and replaces an alias with the canonical table name before the column token is stored in the AST.

3) *Predicate Parsing:* Compound WHERE predicates are parsed by a standard operator-precedence descent:

- 1) `parsePredicateExpr` handles OR (lowest precedence).
- 2) `parseAndExpr` handles AND (higher precedence).
- 3) `parsePrimaryPred` consumes a single leaf comparison `<column> <op> <value>`.

This grammar gives AND higher binding power than OR, consistent with SQL semantics. The result is a PredicateExpr tree in the AST.

B. AST Construction

The AST is defined in `ast.h` as a hierarchy of `std::unique_ptr`-owned nodes under the abstract base `ASTNode`. Table I describes the principal node types.

All ASTNode subtypes are non-copyable and exclusively owned via `unique_ptr`, preventing accidental aliasing.

C. Logical Plan Generation

The `LogicalPlanner` (`logical_planner.cpp`) translates a `QueryNode` into a tree of `LogicalNode` objects by a six-step procedure:

- 1) Validate mandatory AST fields and resolve the primary table in the catalog.

- 2) Create a LogicalScan leaf with a pointer to the catalog's TableMetadata.
- 3) For each explicit JOIN, create a LogicalJoin whose left child is the current plan subtree and right child is a new LogicalScan for the joined table, building a left-deep tree.
- 4) If a WHERE clause is present, wrap the current subtree in a LogicalFilter. Compound predicates use a PredicateTree (the IR analog of PredicateExpr).
- 5) If a GROUP BY clause is present, wrap the subtree in a LogicalAggregate.
- 6) Wrap the final subtree in a LogicalProject with the SELECT column list.

The result is an unoptimized left-deep logical plan tree. For a three-table query, the initial plan is always of the form $\Pi(\sigma_p(\text{Scan}(T_1) \bowtie \text{Scan}(T_2)) \bowtie \text{Scan}(T_3))$, which the CBO will subsequently reorder.

V. IMPORTANT DATA STRUCTURES

A. LogicalNode Hierarchy

The logical plan IR (logical_plan.h) defines six concrete node types under the abstract base LogicalNode:

- **LogicalScan**: leaf; holds table_name and a const pointer to TableMetadata; carries a ScanAlgorithm tag (SequentialScan or IndexScan).
- **LogicalFilter**: unary; holds a BinaryCondition or a PredicateTree for compound predicates; wraps a child node.
- **LogicalJoin**: binary; holds a BinaryCondition for the join predicate and left/right child nodes.
- **LogicalProject**: unary; holds a vector of output column names.
- **LogicalAggregate**: unary; holds GROUP BY column names.
- **LogicalUnion**: binary; combines two subtrees via set union.

Every node implements clone(), dump(), and toJson() to support optimizer restructuring, debugging, and dashboard serialization.

B. PredicateTree

PredicateTree (defined in logical_plan.h) is a recursive AND/OR predicate structure stored inside LogicalFilter for compound WHERE clauses. A leaf node wraps a single BinaryCondition; a branch node holds a logical_op string and two unique_ptr<PredicateTree> children. The method collectLeaves() performs a depth-first traversal to collect all leaf BinaryCondition objects, used by the push-down pass to decompose AND-chains.

C. Catalog and ColumnMetadata

The Catalog class (catalog.h) is a Meyers singleton backed by an unordered_map<string, TableMetadata>. Each TableMetadata records:

- name: table name.
- row_count: total number of rows (used as base cost estimate).
- primary_key: name of the primary-key column.
- columns: map of column name to ColumnMetadata.

ColumnMetadata records:

- name, type (Int, String, Double).
- has_index: whether a B-tree/hash index exists.
- clustered: whether the table is physically sorted by this key.
- distinct_count: used for selectivity estimation.
- min_val, max_val: value range.
- histogram: a vector of HistogramBucket structs (each recording min_val, max_val, count).

The catalog is populated from schema.json at startup via a simple JSON parser built on the nlohmann/json library.

D. PlanStats

PlanStats (optimizer.h) is a value type returned by estimateStats():

```
1 struct PlanStats {
2     double cost;           // total estimated
                           cost
3     double rows;          // estimated output
                           cardinality
4     double cost_scan;     // scan sub-cost
5     double cost_filter;   // filter sub-cost
6     double cost_join;     // join sub-cost
7     double cost_project;  // projection sub-
                           cost
8 };
```

Listing 1. PlanStats structure

Decomposed sub-costs are displayed separately in the dashboard.

E. JoinEdge

During CBO join enumeration, JoinEdge (optimizer.h) records the metadata of each LogicalJoin extracted when flattening the join tree:

```
1 struct JoinEdge {
2     BinaryCondition condition;
3     JoinAlgorithm algorithm;
4     int left_operand_idx;
5     int right_operand_idx;
6 };
```

Listing 2. JoinEdge structure

This records which pair of operand subtrees the original join connected, allowing the DP rebuild phase to recover the correct join conditions.

F. HistogramBucket

HistogramBucket supports range-based selectivity estimation:

```

1 struct HistogramBucket {
2     double min_val;
3     double max_val;
4     std::size_t count;
5 };

```

Listing 3. HistogramBucket structure

Currently, histogram data can be loaded from the catalog JSON but the cost model falls back to simplified selectivity heuristics when histogram data is absent (see Section IX).

VI. RULE-BASED OPTIMIZATION

The rule-based optimizer (RBO) applies two structural transformations before the cost-based pass.

A. Semantic Rewrites

`applySemanticRewrites()` performs a bottom-up traversal of the plan tree and applies two rewrites:

1) *SELECT * Unrolling*: When a `LogicalProject` node carries the wildcard column "*", the optimizer replaces it with the full list of columns from all tables reachable via the subtree (determined by `collect_tables()`). Columns are sorted lexicographically for deterministic output, and qualified as `table_name.col_name`.

2) *OR-to-UNION Rewrite*: When a `LogicalFilter` carries a two-leaf OR predicate directly above a `LogicalScan`, the optimizer expands it into a `LogicalUnion` of two independent filtered scans:

$$\sigma_{p_1 \vee p_2}(R) \longrightarrow \sigma_{p_1}(R) \cup \sigma_{p_2}(R)$$

This rewrite improves opportunities for index utilization in each branch. It applies only when both predicate leaves directly filter the same base table.

B. Predicate Push-Down

`pushDownPredicates()` traverses the plan tree top-down and, when it encounters a `LogicalFilter` directly above a `LogicalJoin`, attempts to push the filter into one of the join's children.

Algorithm 1 describes the procedure:

Column attribution is performed via the `column_in_tables()` helper, which consults the catalog to determine whether a given column name belongs to any table in a set. The helper handles dot-qualified names (e.g., `users.age`) by stripping the table qualifier.

OR predicates are intentionally *not* pushed because the OR condition requires tuples from both branches to be evaluated together; pushing only the left branch would produce incorrect results. The `isPureAndTree()` static method guards against this case.

Algorithm 1 Predicate Push-Down

Require: Plan root N

Ensure: Restructured plan root N'

```

1: procedure PUSHDOWN( $N$ )
2:   Recurse into all children of  $N$  first (bottom-up appli-
   cation)
3:   if  $N = \text{Filter}(p, J)$  and  $J = \text{Join}(L, R, c)$  then
4:      $T_L \leftarrow \text{COLLECTTABLES}(L)$ 
5:      $T_R \leftarrow \text{COLLECTTABLES}(R)$ 
6:     if  $p$  is a pure AND-tree or a leaf predicate then
7:       Decompose  $p$  into conjuncts  $p_1, \dots, p_k$ 
8:       for each  $p_i$  do
9:          $\text{col} \leftarrow$  column referenced in  $p_i$ 
10:        if  $\text{col} \in T_L$  and  $\text{col} \notin T_R$  then
11:          Push  $p_i$  below  $L$ 
12:        else if  $\text{col} \in T_R$  and  $\text{col} \notin T_L$  then
13:          Push  $p_i$  below  $R$ 
14:        else
15:          Keep  $p_i$  above  $J$  as residual filter
16:        end if
17:      end for
18:    else  $\triangleright$  OR predicate: cannot push independently
19:      Return  $N$  unchanged
20:    end if
21:  end if
22:  return restructured plan
23: end procedure

```

VII. COST-BASED OPTIMIZATION

A. Dynamic Programming Join Enumeration

The cost-based optimizer implements System R style subset dynamic programming to enumerate join orderings. The enumeration proceeds over all 2^n subsets of the n join operands, selecting the minimum-cost way to combine any two disjoint subsets into a larger subset. Unlike the original System R (which restricts to left-deep plans), OptiCore explores *all* bushy splits, making it a generalization of the classic algorithm.

1) *Representation*: Subsets of operands are encoded as `uint32_t` bitmasks. Bit i is set if operand i participates in the subset. This limits the implementation to at most 32 operands (practical for OLAP queries with up to ~ 20 tables).

2) *Algorithm*: Algorithm 2 describes the DP enumeration: The Cartesian penalty factor of $100\times$ effectively prunes cross-product joins from the search space while preserving correctness for queries that require them.

3) *Rebuild Phase*: After the DP table is complete, REBUILD recursively reconstructs the plan tree from the (`left_mask`, `right_mask`) pointers stored in each DP entry. During reconstruction, applicable join edges are matched by checking which recorded `JoinEdge` connects a table in the left subtree to a table in the right subtree. The physical join algorithm is selected during this phase (Section VIII).

4) *Complexity*: The DP table has $O(2^n)$ entries. Filling it requires enumerating all subset splits, which costs $O(3^n)$ in

Algorithm 2 System-R Subset DP Join Enumeration**Require:** Operand list $O = [o_0, \dots, o_{n-1}]$, edge set E **Ensure:** Optimal join tree root

```

1: Flatten join tree  $\rightarrow (O, E)$  via FLATTENJOIN
2:  $\text{base}[i] \leftarrow \text{ESTIMATESTATS}(o_i)$  for all  $i$ 
3: for  $i = 0$  to  $n - 1$  do
4:    $dp[1 \ll i] \leftarrow (\text{base}[i].\text{cost}, \text{base}[i].\text{rows}, \text{leaf} = i)$ 
5: end for
6: for  $k = 2$  to  $n$  do
7:   for each  $S \subseteq [n]$  with  $|S| = k$  do
8:     for each split  $S_1, S_2$  with  $S_1 \cup S_2 = S, S_1 \cap S_2 = \emptyset, S_1 < S_2$  do
9:        $\text{hasEdge} \leftarrow \exists e \in E : e \text{ connects } S_1 \text{ and } S_2$ 
10:       $r_{\text{join}} \leftarrow dp[S_1].\text{rows} \times dp[S_2].\text{rows} \times$ 
       $\begin{cases} 0.01 & \text{hasEdge} \\ 1.0 & \text{otherwise} \end{cases}$ 
11:       $C_{NL} \leftarrow dp[S_1].\text{cost} + dp[S_2].\text{cost} +$ 
       $dp[S_1].\text{rows} \times dp[S_2].\text{rows}$ 
12:       $C_{HJ} \leftarrow dp[S_1].\text{cost} + dp[S_2].\text{cost} +$ 
       $dp[S_1].\text{rows} + dp[S_2].\text{rows}$ 
13:       $C_{MJ} \leftarrow dp[S_1].\text{cost} + dp[S_2].\text{cost} + C_{\text{sort}} +$ 
       $dp[S_1].\text{rows} + dp[S_2].\text{rows}$ 
14:      if no edge: multiply all costs by 100 then
15:         $C_{\text{join}} \leftarrow \min(C_{NL}, C_{HJ}, C_{MJ})$ 
16:        if  $C_{\text{join}} < dp[S].\text{cost}$  (or  $dp[S]$  undefined)
17:          then
18:             $dp[S] \leftarrow (C_{\text{join}}, r_{\text{join}}, S_1, S_2)$ 
19:          end if
20:
21:
22:      return REBUILD( $dp, (1 \ll n) - 1$ )

```

total (each element is in either the left subset, the right subset, or neither subset). For each split, edge detection requires $O(n \cdot |E|)$ work. Total complexity: $O(3^n \cdot n \cdot |E|)$, practical for $n \leq 20$.

B. Join Reordering

The `optimizeJoinOrder()` entry point traverses the plan tree, passes transparently through Project, Filter, and Aggregate wrappers, and dispatches any Join root to the DP engine. This allows multi-join sub-trees nested inside projections to be reordered independently.

VIII. PHYSICAL OPERATOR SELECTION

For each join pair (S_1, S_2) , OptiCore evaluates three physical algorithms using the formulas below and selects the minimum-cost option.

A. Nested Loop Join

Nested Loop Join (NLJ) iterates the outer relation once and scans the inner relation for every outer tuple. Its estimated cost is:

$$C_{NL}(L, R) = C(L) + C(R) + |L| \cdot |R| \quad (1)$$

where $C(\cdot)$ is the recursive plan cost and $|\cdot|$ is the estimated output cardinality. NLJ is selected for non-equi-join predicates (e.g., inequality comparisons) since hash-based methods require equality conditions.

B. Hash Join

Hash Join builds an in-memory hash table on the smaller relation and probes it with each tuple from the larger relation. Its estimated cost is:

$$C_{HJ}(L, R) = C(L) + C(R) + |L| + |R| \quad (2)$$

Hash Join is preferred over NLJ when estimated cardinalities are large, because $\|L| + |R|\| \ll |L| \cdot |R|$ for large intermediate result sets.

C. Merge Join

Merge Join requires both inputs to be sorted on the join key. The sort overhead is modeled as:

$$C_{\text{sort}} = |L| \log_2(|L|) + |R| \log_2(|R|) \quad (3)$$

Total Merge Join cost:

$$C_{MJ}(L, R) = C(L) + C(R) + C_{\text{sort}} + |L| + |R| \quad (4)$$

Merge Join is only considered for equi-join predicates. It outperforms Hash Join when inputs are already partially sorted or when memory pressure prohibits hashing.

D. Algorithm Selection Policy

The selection logic applied during the DP rebuild phase is:

- 1) If the join condition is an equality predicate (`op = "="`): compare C_{MJ} , C_{HJ} , C_{NL} and select the minimum.
- 2) If the join condition is a non-equality predicate (or absent): always select NLJ, as hash- and merge-based algorithms require equality conditions.

IX. COST MODEL

The cost model is implemented in `Optimizer::estimateStats()`. All cost quantities are in abstract “row-read” units that compare consistently across node types.

A. Scan Cost

$$C_{\text{seq}}(T) = N_T, \quad C_{\text{idx}}(T) = 5 \cdot \log_2(N_T + 2) \quad (5)$$

where N_T is the table’s row count from the catalog. Index scan cost is modeled as logarithmic (approximating B-tree traversal) and is selected when the `LogicalScan` node carries an `IndexScan` algorithm tag.

B. Filter Selectivity and Cost

Selectivity estimation is the process of predicting what fraction of tuples pass a predicate. OptiCore’s selectivity function `computeSelectivity()` uses the following rules:

- 1) **Equality** ($op = "="$): if the column has a known distinct count d , selectivity is $1/d$; otherwise the default is 0.10.
- 2) **Range** ($op \in \{<, >, \leq, \geq\}$): if histogram buckets are available, the fraction of bucket counts within the predicate range is computed; otherwise the default is 0.25.
- 3) **Inequality** ($op = "!="$): $1 - 1/d$ if distinct count is known, else 0.90.
- 4) **Compound AND**: $sel(p_1) \cdot sel(p_2)$ (independence assumption).
- 5) **Compound OR**: $sel(p_1) + sel(p_2) - sel(p_1) \cdot sel(p_2)$.

Filter cost:

$$C_{\text{filter}}(F) = C(\text{child}(F)) + 0.2 \cdot |\text{child}(F)| \quad (6)$$

Estimated output cardinality:

$$|F| = sel(p) \cdot |\text{child}(F)| \quad (7)$$

C. Join Cost

$$|J| = \max(1, |L| \cdot |R| \cdot sel_J) \quad (8)$$

where $sel_J = 0.01$ for equijoins and $sel_J = 1.0$ for cross joins.

$$C_{\text{join}}(J) = C(L) + C(R) + |J| \quad (9)$$

D. Projection Cost

$$C_{\text{proj}}(\Pi) = C(\text{child}(\Pi)) + 0.5 \cdot |\text{child}(\Pi)| \quad (10)$$

E. Aggregate Cost

$$C_{\text{agg}}(\gamma) = C(\text{child}(\gamma)) + 1.5 \cdot |\text{child}(\gamma)| + |\gamma| \quad (11)$$

$$|\gamma| = \max(1, 0.2 \cdot |\text{child}(\gamma)|)$$

X. DASHBOARD AND VISUALIZATION

OptiCore includes an interactive HTML5 dashboard (`optimizer_dashboard.html`, generated by `html_reporter.cpp`) that displays:

- **Plan Trees**: Before (unoptimized) and after (optimized) plan trees rendered as interactive JSON-driven node graphs.
- **Cost Breakdown**: Pie chart showing the proportion of scan, filter, join, and projection costs in the optimized plan.
- **Reconstructed SQL**: The `SqlReconstructor` visitor traverses the optimized plan tree and emits an equivalent SQL string with sub-selects reflecting the actual physical evaluation structure.
- **Execution Trace**: Row-level output from the Volcano-style executor showing simulated tuple-at-a-time data flow.

A. Dashboard Architecture

The dashboard is driven by a zero-dependency Python microservice (`server.py`) that bridges the browser’s Fetch API to the C++ optimizer binary. The workflow is:

- 1) The browser sends a POST request with the SQL string to the Python server’s `/optimize` endpoint.
- 2) The server spawns the C++ binary as a subprocess and pipes the SQL string to its stdin.
- 3) The binary runs the full optimizer pipeline and emits a JSON document to stdout containing the AST dump, logical plan (before), optimized plan (after), cost breakdown, and reconstructed SQL.
- 4) The Python server relays the JSON to the browser; the dashboard renders it via vanilla JavaScript without any frontend framework.

B. SQL Reconstructor

The `SqlReconstructor` (`sql_reconstructor.cpp`) implements a recursive visitor pattern over the `LogicalNode` tree. Each node type produces a SQL fragment:

- **LogicalScan**: emits `table_name`.
- **LogicalFilter**: wraps the child in a correlated subselect with a WHERE clause derived from the predicate.
- **LogicalJoin**: emits an explicit `JOIN ... ON` clause.
- **LogicalProject**: wraps the subtree in a `SELECT ... FROM`.
- **LogicalUnion**: emits a UNION between two branches.

The depth parameter is used to generate fresh aliases (`T0`, `T1`, ...) for nested subselects.

XI. IMPLEMENTATION DETAILS

A. Technology Stack

OptiCore is implemented as a multi-tier system spanning three layers. The **core optimization engine** is written in C++17, comprising approximately 6,400 lines of source code across 15 files. The **microservice bridge** is a zero-dependency Python 3 HTTP server (`server.py`) that manages subprocess communication between the browser and the C++ binary. The **visualization frontend** is built entirely in HTML5, CSS3, and vanilla JavaScript with no framework dependencies, ensuring portability and ease of study.

B. Module Organization

The implementation is divided into the following modules:

- **Parser** (`parser.cpp`, 292 lines): tokenizer and recursive-descent SQL grammar.
- **Logical Planner** (`logical_planner.cpp`, 135 lines): AST-to-logical-plan translation.
- **Rule-Based Optimizer** (`optimizer.cpp`, 1,022 lines): semantic rewrites and predicate push-down.
- **Cost-Based Optimizer** (`optimizer.cpp`): System R DP join enumeration and physical operator selection, included in the same translation unit.

TABLE II
CODEBASE STATISTICS

Attribute	Value
Core language	C++17
Frontend	HTML5, CSS3, Vanilla JavaScript
Backend bridge	Python 3 (zero-dependency)
Total source lines	≈6,400 (src + headers + tests)
Optimization modules	2 (RBO + CBO)
Supported join algorithms	3 (NLJ, Hash Join, Merge Join)
Rewrite rules	3 (Predicate Push-Down, OR-to-UNION, SELECT* Unrolling)
Dashboard	Interactive HTML5 with JSON-driven plan trees
Parser	Recursive-descent, hand-written
Logical plan nodes	6 (Scan, Filter, Join, Project, Aggregate, Union)
Cost estimator	Analytical model with histogram support
Unit tests	≈30 tests across parser, catalog, RBO, CBO

- **Cost Estimator** (`optimizer.h/.cpp`): `estimateStats()` implementing scan, filter, join, projection, and aggregate cost formulas.
- **Catalog** (`catalog.cpp`, 109 lines): singleton catalog backed by `nlohmann/json` schema loading.
- **Physical Plan Generator**: integrated into the DP rebuild phase of the CBO; selects NLJ, HJ, or MJ per join pair.
- **SQL Reconstructor** (`sql_reconstructor.cpp`, 108 lines): recursive visitor emitting SQL from the optimized plan.
- **Dashboard Generator** (`html_reporter.cpp`, 889 lines): serializes the full pipeline output to JSON and HTML.
- **Executor** (`executor.cpp`, 384 lines): Volcano-style iterator model for simulated tuple-at-a-time execution.

C. Module Interaction

The modules interact through a strict unidirectional data flow. `main.cpp` reads the SQL query from `stdin`, invokes the parser, and passes the resulting AST to the logical planner. The planner calls the catalog singleton to resolve table and column references. The optimizer receives the logical plan and applies RBO and CBO passes in sequence, consulting the catalog for cardinality and selectivity estimation. The resulting physical plan is passed to both the SQL reconstructor (for textual output) and the dashboard generator (for HTML/JSON output), which writes the report to `stdout` for relay by the Python server.

D. Codebase Summary

Table II summarizes key implementation statistics.

XII. COMPLEXITY ANALYSIS

Table III summarizes the theoretical time complexity of each pipeline stage.

The dominant cost of the optimizer pipeline is the CBO’s join enumeration step. The subset DP algorithm explores all 2^n subsets of n join operands. For each subset of size k , it enumerates all splits into two disjoint non-empty subsets, which

TABLE III
ALGORITHMIC COMPLEXITY OF PIPELINE STAGES

Stage	Complexity	Notes
Parsing	$O(L)$	L = query string length
AST construction	$O(T + P)$	T = tables, P = predicates
Logical planning	$O(T)$	One scan node per table
Predicate push-down	$O(N \cdot k)$	N = plan nodes, k = conjuncts
Join enumeration	$O(3^n \cdot n \cdot E)$	n = join operands
Cost estimation	$O(N)$	One pass over plan tree
SQL reconstruction	$O(N)$	One pass over plan tree

is equivalent to enumerating all ordered partitions of the subset into two parts. The total number of (subset, split) pairs across all subset sizes is $\sum_{k=0}^n \binom{n}{k} \cdot 2^{k-1} = 3^n/2$, yielding the well-known $O(3^n)$ bound for the enumeration loop. For each split, edge detection requires a scan over the $|E|$ join edges for each of the n operands, contributing the $n \cdot |E|$ factor. In practice, $|E| \leq \binom{n}{2}$, so the worst-case complexity is $O(3^n \cdot n^3)$.

This exponential growth makes the exhaustive DP approach impractical for more than approximately 20 join operands. For small OLAP queries with 3–10 tables, however, the algorithm completes in microseconds and provides provably optimal join orderings within the cost model’s assumptions.

XIII. EXPERIMENTAL EVALUATION

We evaluate OptiCore on five representative benchmark workloads that cover the major optimizer capabilities. All benchmarks use the catalog defined in `schema.json`, which contains three core benchmark tables: `users` (100,000 rows), `orders` (1,000,000 rows), and `products` (50,000 rows).

A. Benchmark Queries

Q1 — Simple Filter Push-Down

```
1 SELECT * FROM users WHERE age > 30;
```

Listing 4. Q1: Single-table filter

Q2 — Two-Table Equijoin

```
1 SELECT users.name, orders.total
2 FROM users
3 JOIN orders ON users.id = orders.user_id
4 WHERE orders.total > 4000;
```

Listing 5. Q2: Two-table join with filter

Q3 — Three-Table Star Join

```
1 SELECT * FROM users u
2 JOIN orders o ON u.id = o.user_id
3 JOIN products p ON o.product_id = p.id
4 WHERE u.age > 30 AND o.total > 1500
5 AND p.price < 500;
```

Listing 6. Q3: Three-table star schema join

Q4 — Non-Equijoin Filter

```
1 SELECT users.name, orders.total
2 FROM users
3 JOIN orders ON users.id = orders.user_id
```


TABLE IV
ESTIMATED PLAN COST COMPARISON (ABSTRACT ROW-READ UNITS)

Query	Initial Cost	Optimized Cost	Reduction
Q1 (Filter)	1.33×10^5	1.33×10^5	0.0%
Q2 (2-Table Join)	1.33×10^9	3.76×10^8	71.6%
Q3 (Star Join)	6.10×10^{11}	2.82×10^{10}	95.4%
Q4 (Cross-Col. Filter)	1.33×10^9	1.33×10^9	0.0%
Q5 (OR-to-UNION)	1.36×10^5	2.57×10^5	-89.6% [†]

[†]OR-to-UNION trades lower cost for two separate scans, enabling future per-branch index utilization.

TABLE V
JOIN ALGORITHM SELECTED BY CBO

Query / Join Pair	Algorithm	Est. Join Rows
Q2 (users $\bowtie$ filtered orders)	Hash Join	2.50×10^8
Q3 (users $\bowtie$ orders, after push-down)	Hash Join	2.50×10^7
Q3 (result $\bowtie$ products)	Hash Join	1.00×10^{10}
Q4 (users $\bowtie$ orders, equijoin)	Hash Join	1.00×10^9

TABLE VI
SELECTIVITY ESTIMATION FOR FILTER PREDICATES

Predicate	Selectivity	Output Rows
users.age > 30	0.25 (range default)	25,000
orders.total > 1500	0.10 (histogram)	100,000
products.price < 500	0.80 (histogram)	40,000
orders.total > 4000	0.25 (range default)	250,000
age < 18 OR age > 65	0.31 (histogram)	31,006

```
4 WHERE orders.total > users.id;
```

Listing 7. Q4: Equijoin with cross-column WHERE filter

Q5 — OR-to-UNION Rewrite

```
1 SELECT name FROM users
2 WHERE age < 18 OR age > 65;
```

Listing 8. Q5: OR predicate triggering UNION rewrite

B. Benchmark Results

Tables IV–VI report values derived from the optimizer’s cost model using the catalog statistics in `schema.json`. All costs are in abstract row-read units as defined by the cost model (Section IX).

C. Q3 Reference Measurement

The README provides a verified reference measurement for Q3. The initial plan cost (without any optimization) is reported as 6.10×10^{11} and the optimized plan cost after predicate push-down and join reordering is 2.82×10^{10} , yielding a cost reduction of approximately 95.4%. This reduction arises from three cumulative effects: (1) predicate push-down reduces intermediate cardinality before joining by a factor of approximately $8 \times$ per table; (2) join reordering places the smallest filtered tables at the innermost positions, compounding

the cardinality reduction; and (3) Hash Join replaces Nested Loop Join for all high-cardinality equijoins, reducing join cost from quadratic to linear in the input sizes.

D. Discussion

The benchmark results illuminate how each optimization technique contributes to overall cost reduction.

Simple queries (Q1) show no improvement because a single-table filter query has no joins to reorder and no predicates to push through a join node. The optimizer correctly recognizes this and leaves the plan unchanged. This behavior is desirable: a production optimizer must avoid degrading already-optimal plans.

Multi-join queries (Q2, Q3) benefit most significantly. For Q2, pushing the `orders.total > 4000` filter below the join reduces the orders scan cardinality from 1,000,000 to 250,000 before joining, reducing the join cost by 71.6%. For Q3, the effect compounds across three tables: predicate push-down reduces users, orders, and products to 25,000, 100,000, and 40,000 rows respectively before any joins are executed.

Predicate push-down provides the primary cardinality reduction, while join reordering provides secondary improvement by ensuring that the two smallest filtered sets are joined first. In Q3, the optimal order identified by the DP algorithm joins the filtered users and filtered orders first (producing a small intermediate set) before joining with products.

The non-equijoin filter in Q4 cannot be pushed below the join because the predicate `orders.total > users.id` references columns from both tables. With only two tables, there is also no join reordering opportunity. The optimizer correctly identifies both conditions and leaves the plan unchanged, demonstrating conservative but correct behavior.

The OR-to-UNION rewrite in Q5 increases raw estimated cost because it duplicates the scan of the `users` table. However, this structural transformation is semantically valuable: by splitting the OR into two independent branches, each branch can independently exploit an index scan if one were available on `users.age`. Under the current catalog (no index on `age`), both branches perform sequential scans, and the cost increase reflects this correctly. The rewrite is therefore an investment in future index-aware optimization rather than an immediate cost reduction.

Dynamic programming becomes increasingly valuable as query complexity grows. For two-table queries, there is at most one join ordering to consider. For three tables, there are three possible orderings (ignoring commutativity), and the DP correctly identifies the minimum-cost one. For n tables, the number of bushy plan alternatives grows super-exponentially, and the DP ensures that the global optimum is found within the cost model’s assumptions.

XIV. FEATURE COMPARISON

Table VII compares OptiCore with PostgreSQL and MySQL across high-level optimizer capabilities.

TABLE VII
FEATURE COMPARISON: OPTICORE VS. PRODUCTION OPTIMIZERS

Feature	OptiCore	PostgreSQL	MySQL
Predicate Push-Down	✓	✓	✓
Join Reordering	✓	✓	✓
Cost-Based Optimization	✓	✓	✓
Histogram Statistics	Partial [†]	✓	✓
Index-Aware Costing	Partial [‡]	✓	✓
Subquery Optimization	—	✓	✓
Outer Join Support	—	✓	✓
Visualization Dashboard	✓	—	—
Educational Transparency	✓	—	—
Modular / Extensible	✓	Partial	Partial

[†]Histogram data is loaded but selectivity falls back to heuristics.

[‡]Index scan is costed as $O(\log N)$ but not auto-promoted.

XV. DISCUSSION

A. Strengths

OptiCore’s primary strength lies in its *completeness within scope*: it implements the full classical optimizer pipeline end-to-end, from tokenization through plan visualization, with no external database dependencies. Every component is independently testable, and the unit-test suite covers all major pipeline stages. The modular architecture allows individual components to be studied, modified, or replaced without affecting the rest of the system, making it suitable for course assignments and research experiments.

B. Practical Usefulness

Beyond its educational role, OptiCore demonstrates that the core ideas of cost-based query optimization can be implemented in approximately 6,400 lines of well-structured C++ code. The three-table star-join benchmark achieves a 95.4% estimated cost reduction, which validates that the cost model correctly captures the dominant factors in join cost: intermediate cardinality and algorithm selection. The interactive dashboard makes these optimizations immediately visible, providing rapid feedback for understanding the effect of different catalog statistics on optimizer behavior.

C. Educational Importance

The visualization dashboard is the feature that most distinguishes OptiCore from academic implementations of query optimization algorithms. Students can observe the exact moment at which predicate push-down restructures the plan tree, inspect the cost breakdown of each node before and after optimization, and compare the reconstructed SQL of the unoptimized and optimized plans. This transparency addresses a fundamental pedagogical challenge: query optimization theory is well-understood, but the gap between theory and implementation is large. OptiCore bridges this gap by providing a working implementation that is also fully observable.

D. Optimization Effectiveness

The experimental results confirm that predicate push-down and join reordering are highly effective and complementary techniques. Push-down reduces per-table cardinality before joins are applied, which in turn makes the DP cost estimations more accurate and improves join ordering decisions. The OR-to-UNION rewrite is structurally sound but does not reduce cost under the current sequential-scan model; its value becomes apparent when index-aware costing is added.

E. Modularity and Extensibility

The optimizer is organized as a pipeline of independently invocable passes. New rewrite rules can be added to the RBO by implementing an additional traversal in `optimizer.cpp`. New physical operators can be added to the CBO’s rebuild phase by extending the algorithm selection policy. The catalog’s support for histogram buckets and index flags provides infrastructure for more sophisticated cost models without architectural changes.

XVI. KNOWN LIMITATIONS

OptiCore is an educational implementation. The following limitations are known and expected:

- 1) **Limited SQL grammar.** Arithmetic expressions in SELECT, HAVING clauses, ORDER BY, LIMIT, OFFSET, DISTINCT, and window functions are not supported. The grammar covers only the subset of SQL needed to demonstrate the core optimizer pipeline.
- 2) **No correlated subqueries.** Correlated or uncorrelated subqueries in SELECT, FROM, or WHERE are not parsed or optimized. A decorrelation pass (as in the Unnesting Arbitrary Queries technique [2]) would be necessary to support this feature.
- 3) **Limited aggregation support.** The LogicalAggregate node is planned and costed, but there is no aggregate push-down or grouping rewrite. GROUP BY is parsed and included in the plan tree but is not subject to optimization.
- 4) **No histogram statistics integration.** The catalog schema supports HistogramBucket data, and histogram buckets are loaded from `schema.json`, but the selectivity estimator falls back to simplified heuristics for most predicate types. Full histogram-based estimation would improve cost accuracy on skewed distributions.
- 5) **Simplistic cardinality estimation.** The cost model assumes column independence (the product of selectivities for AND predicates) and uses fixed default selectivities (0.25 for range, 0.10 for equality). These assumptions can be inaccurate on highly correlated or skewed data.
- 6) **No adaptive optimization.** The optimizer produces a static plan before execution. Runtime feedback (observed row counts at key pipeline stages) is not used to re-optimize mid-execution.
- 7) **No runtime re-optimization.** Related to the above, there is no mechanism to detect when cost-model estimates

diverge significantly from runtime observations and trigger re-planning.

- 8) **No index advisor.** The optimizer notes when index scans would be beneficial but does not suggest index creation or automatically promote sequential scans to index scans based on predicate selectivity.
- 9) **No transaction-aware optimization.** The optimizer does not account for transaction isolation levels, lock contention, or multi-version concurrency control overhead, all of which affect practical plan selection in production systems.
- 10) **No concurrency modeling.** The cost model assumes single-threaded execution. Parallel query optimization (exchange operators, partition pruning) is not modeled.
- 11) **Exponential DP search space.** The $O(3^n)$ join enumeration becomes impractical for $n > 20$ operands. Production systems use heuristic pruning (e.g., restricting to left-deep plans or using genetic algorithms) to handle larger join sets.
- 12) **Limited physical operators.** Only three join algorithms (NLJ, Hash Join, Merge Join) are supported. Index Nested Loop Join, Grace Hash Join, and sort-merge variants with spilling are not implemented.
- 13) **No self-join support.** A relation may appear at most once in a query. Queries such as `FROM users u JOIN users m ON u.manager_id = m.id` are not supported because the system deduplicates operands by table name during DP flattening.
- 14) **INNER JOIN only.** LEFT, RIGHT, FULL OUTER JOIN, SEMI JOIN, and ANTI JOIN are not implemented.
- 15) **No column-lineage tracking.** Column attribution during push-down uses catalog lookup rather than lineage tracking, which can fail for derived columns or aliased expressions.
- 16) **No multi-pass catalog updates.** The catalog is loaded once at startup; there is no mechanism to update statistics during a session or between queries.

XVII. FUTURE WORK

Several extensions would meaningfully increase the realism and utility of OptiCore:

- 1) **Cascades-style top-down search.** Replacing the bottom-up DP enumeration with a top-down memoized search (as in the Cascades framework [3]) would provide a cleaner extension point for adding new physical operators and equivalence rules, and would enable goal-directed pruning of the search space.
- 2) **Memoization and plan recycling.** The current DP table is recomputed for each query. A persistent memo structure would allow plan fragments to be reused across queries sharing common sub-expressions, as in the Cascades framework.
- 3) **Adaptive query optimization.** Runtime feedback (row counts observed at key pipeline stages) could be used to re-optimize mid-execution, addressing the limitation of pre-execution cost models on skewed data distributions.

- 4) **Machine-learning cost models.** Replacing the analytical cost model with a learned model trained on observed execution times would allow OptiCore to adapt to workload-specific cost characteristics without hand-tuning cost constants.
- 5) **Busby join enumeration with pruning.** The current implementation explores all bushy splits, which is correct but expensive. Restricting to left-deep plans with heuristic ordering (e.g., smallest table first) would allow larger join sets to be optimized while remaining pedagogically useful.
- 6) **Parallel query optimization.** An exchange operator could be inserted above scans and joins to model intra-query parallelism and allow OptiCore to estimate multi-core execution costs.
- 7) **Distributed query optimization.** Extending the cost model with network transfer costs and partition statistics would allow OptiCore to model distributed query execution, relevant to modern data warehouse architectures.
- 8) **Vectorized execution.** Replacing the Volcano iterator model with a vectorized (column-at-a-time) execution model would demonstrate the interaction between physical plan structure and modern CPU cache behavior.
- 9) **Runtime statistics collection.** Instrumenting the executor to collect actual row counts, join selectivities, and scan costs at runtime would close the feedback loop between execution and cost estimation, enabling automatic calibration of the cost model.
- 10) **Index recommendation.** Analyzing the predicates and join conditions present in a workload and recommending index structures that would most reduce total estimated cost would extend OptiCore into the space of physical design advisors.
- 11) **Histogram-based cardinality estimation.** The infrastructure for per-column `HistogramBucket` data is already present in the catalog; integrating it fully into `computeSelectivity()` would significantly improve cost estimates on skewed distributions.
- 12) **Outer join support.** Extending the join enumeration to handle LEFT/RIGHT/FULL OUTER JOIN with correct null-handling semantics would make OptiCore suitable for a broader class of analytical queries.

XVIII. CONCLUSION

This paper presented OptiCore, a self-contained, pedagogically transparent SQL query optimizer implemented in C++17 that covers the complete classical optimizer pipeline from tokenization and recursive-descent parsing, through logical plan construction, rule-based predicate push-down and OR-to-UNION rewriting, System R style subset dynamic programming join enumeration, and physical operator selection, to interactive HTML dashboard visualization.

The hybrid two-phase optimization architecture demonstrates how rule-based and cost-based techniques complement each other in practice. The rule-based predicate push-down pass reduces intermediate cardinality before joins are executed;

the cost-based dynamic programming pass then identifies the globally optimal join ordering within the reduced search space. Together, these two passes achieve a verified 95.4% cost reduction on the three-table star-join benchmark, and a 71.6% reduction on the two-table equijoin benchmark with filter push-down.

The visualization dashboard represents OptiCore’s most distinctive pedagogical contribution: it makes every optimizer decision immediately observable, displaying before- and after-optimization plan trees, cost breakdowns per node, and the reconstructed SQL of the optimized plan. Students can observe the exact effect of predicate push-down on the plan tree, inspect the DP table’s cost estimates for different join orderings, and understand how physical algorithm selection (Hash Join vs. Merge Join vs. Nested Loop Join) interacts with cardinality estimates.

The system is implemented in approximately 6,400 lines of well-structured C++17 across 15 source files, with a comprehensive unit-test suite covering all major pipeline stages. Its modular, layered architecture makes individual components

easy to study, extend, and modify, making OptiCore a practical resource for database systems courses, graduate research, and independent study of relational query optimization internals.

REFERENCES

- [1] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD International Conference on Management of Data*, Boston, MA, USA, 1979, pp. 23–34.
- [2] G. Graefe, “Volcano — An extensible and parallel query evaluation system,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 6, no. 1, pp. 120–135, Feb. 1994.
- [3] G. Graefe and W. J. McKenna, “The Cascades framework for query optimization,” *IEEE Data Engineering Bulletin*, vol. 18, no. 3, pp. 19–29, 1995.
- [4] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY, USA: McGraw-Hill, 2003.
- [5] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.
- [6] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” <https://www.postgresql.org/docs/>, 2024, accessed June 2026.
- [7] Carnegie Mellon University Database Group, “CMU 15-445/645: Database Systems,” <https://15445.courses.cs.cmu.edu/>, 2024.
- [8] Y. E. Ioannidis, “Query optimization,” *ACM Computing Surveys*, vol. 28, no. 1, pp. 121–123, Mar. 1996.